

SENTINELNET – AI-POWERED NETWORK INTRUSION DETECTION SYSTEM (NIDS)

Author: Chaganti Sai Sarvani

Mentor: Dr. N Jagan Mohan

ABSTRACT

SentinelNet is an advanced AI-powered Network Intrusion Detection System designed to detect and classify cyber threats in real-time, leveraging two benchmark datasets, NSL-KDD and CICIDS2017. Comprehensive data preprocessing, feature engineering, and machine learning models—Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting—are employed for both binary and multi-class classification. Experimental results highlight the robustness of ensemble approaches in identifying diverse network threats. This document details dataset descriptions, methodologies, experiments, and analysis.

KEYWORDS

Network Intrusion Detection System, NSL-KDD, CICIDS2017, Machine Learning, Ensemble Methods, Binary Classification, Multi-class Classification, Cybersecurity, Anomaly Detection, Decision Tree, Random Forest, Support Vector Machine, Feature Engineering.

1. INTRODUCTION

Network security is critical in protecting organizational data and infrastructure against cyber threats. Intrusion Detection Systems (IDS) monitor network traffic to identify malicious activities. Traditional signature-based IDS face challenges detecting novel attacks, motivating AI-powered IDS leveraging machine learning for adaptive threat detection.

With the exponential rise in cyber threats targeting networks worldwide, traditional signature-based intrusion detection systems face limitations in detecting novel and sophisticated attacks. AI and machine learning offer dynamic, adaptive techniques for identifying anomalous network behavior with greater accuracy. The SentinelNet project aims to design an intelligent NIDS system based on supervised machine learning algorithms trained on the NSL-KDD dataset representing various attack types and normal traffic. The goal is to improve detection rates and provide timely alerts to network administrators.

This document presents SentinelNet, a machine learning based NIDS evaluated using NSL-KDD and CICIDS2017 datasets. The system classifies network traffic to detect intrusions and categorize attack types, aiding network administrators in timely responses.

2. LITERATURE REVIEW

2.1 Evolution of Intrusion Detection System (IDS)

- IDS started as rule-based systems relying on fixed attack signatures.
- Signature-based systems work well for known attacks but fail to detect new/unknown threats (zero-day attacks)
- Anomaly-based IDS was introduced to detect unusual network behavior by learning normal traffic patterns.
- Anomaly detection can find unknown attacks, but tends to have a higher false positive rate.

2.2 Common Benchmark Datasets for IDS Research

- **KDD Cup 1999**
 - One of the earliest and most widely used IDS benchmark datasets.
 - Criticized for redundancy, outdated attack types, and imbalance in data, which may bias model evaluation.
- **NSL-KDD**
 - An Refined and improved version of the original KDD Cup 1999 dataset.
 - Removes duplicate records, balances classes better, and provides a more realistic evaluation scenario for IDS.
- **CICIDS2017**
 - Reflects realistic modern enterprise network traffic.
 - Includes diverse and contemporary attack types like Distributed Denial of Service (DDoS), infiltration and web exploits.
 - Considered more representative of current cybersecurity challenges.

2.3 Machine Learning Algorithms Widely Used for Intrusion Detection

- **Decision Trees:** Simple, easy to interpret classification models.
- **Random Forests:** Ensemble of decision trees for improved accuracy and stability.
- **Support Vector Machines (SVM):** Effective for high-dimensional data classification.
- Ensemble Methods combining multiple classifiers
- Hybrid IDS models that leverage strengths of both signature and anomaly approaches

2.4 Insights Gained from Literature

- **Data Preprocessing and Feature Selection:** Essential to remove irrelevant or noisy features to improve model accuracy.
- **Class Imbalance Problem:** Some attack types are underrepresented, leading to biased models.
- **Trade-offs:** Signature-based IDS is fast but less adaptive; anomaly-based IDS is adaptive but can cause more false alarms.

- **Hybrid Approaches:** Effective systems combine both to benefit from high-speed detection and adaptive learning.

3. METHODOLOGY

3.1 Dataset Description

3.1.1 NSL-KDD Dataset

- NSL-KDD is a benchmark dataset aimed at providing a realistic basis for training and evaluating intrusion detection models. It improves upon the original KDD Cup 1999 dataset by eliminating redundant records and addressing class imbalance concerns.
- **Features:** It features 41 attributes encompassing basic connection characteristics (protocol type, service type, source and destination bytes) alongside traffic-level details like connection flags and error rates.
- **Volume:** Training data includes 125,973 records, whereas testing data contains 22,544 records.
- **Attack Types:** It covers four broad classes of attacks:
 - Denial of Service (DoS): includes attacks like smurf, neptune
 - Probing: e.g., portsweep, nmap
 - User to Root (U2R): e.g., buffer overflow, rootkit
 - Remote to Local (R2L): e.g., ftp write, guess password

3.1.2 CICIDS2017 Dataset

- CICIDS2017 represents a more contemporary and realistic dataset simulating enterprise network traffic including normal activities and various sophisticated attacks.
- **Features:** Contains between 78 to 83 features per record derived from flow and packet-level statistics.
- **Volume:** Over 2.8 million labeled records collected across five consecutive days reflect changing and dynamic network behavior.
- **Attack Types:** Includes a variety of modern attacks such as:
 - Distributed DoS attacks like Hulk, GoldenEye, Slowloris
 - Brute force login attempts
 - Web application attacks (SQL injection, Cross-Site Scripting)
 - Botnet activity and infiltration techniques
 - Reconnaissance attacks like PortScan and Heartbleed

3.1.3 Dataset Comparison

NSL-KDD is suitable for lightweight model evaluation and academic benchmarking, while CICIDS2017 better reflects real-world scenarios and supports practical deployment testing. The choice of dataset depends on the research goal—NSL-KDD for foundational model testing, CICIDS2017 for advanced, scalable intrusion detection system development. CICIDS2017's larger size and modern attack variety provide a more comprehensive training ground for sophisticated AI models. However, NSL-KDD remains valuable for initial experimentation due to its simplicity and ease of use.

Attribute	NSL-KDD	CICIDS2017
Number of Features	41	78-83
Total Records	~150,000	>2.8 million
Traffic Type	Simulated, older generation	Realistic, enterprise scale
Typical Usage	Lightweight model evaluation	Practical deployment
Attack Variety	4 legacy attack categories	14+ modern attack classes

Table 1: Dataset Comparison

3.2 Dataset Preprocessing

An accurate IDS requires clean, balanced, and meaningful data. The preprocessing pipeline includes:

- **Dealing with Missing Data:** Records with missing or corrupt data points are either corrected through imputation or removed to preserve dataset integrity.
- **Encoding Categorical Data:** Categorical variables such as protocol type, service, and connection flags are converted to numerical format via one-hot encoding for compatibility with machine learning algorithms.
- **Feature Scaling:** Numerical data is standardized using z-score normalization (StandardScaler) to ensure features contribute proportionally during training and to accelerate convergence in learning algorithms.
- **Correlation Analysis:** Pairwise correlation metrics identify redundant features with excessive similarity. Highly correlated features are discarded to reduce noise and multicollinearity.
- **Handling Class Imbalance:** Attack classes are often underrepresented compared to normal data, which can bias models. Mitigation techniques include:
 - Undersampling the majority (normal) class to reduce dominance
 - Power sampling minority classes to enhance their representation without generating synthetic data, preserving dataset realism
- **Data Splitting:** A stratified train-test split maintains class distribution proportionality, ensuring meaningful evaluation that reflects actual deployment scenarios.

3.2.1 Scaling vs Normalization

Standardization transforms features to have a mean of 0 and a standard deviation of 1, making it suitable for algorithms assuming Gaussian distributions like SVM and logistic regression. Normalization rescales features to the range, which benefits bounded-input algorithms such as neural networks and KNN, but is sensitive to outliers.

Aspect	Standardization	Normalization	Relevance to NSL-KDD	Relevance to CICIDS2017
Definition	Transforms data to have mean = 0, std = 1	Rescales data to range	Used to handle Gaussian-distributed features; helps SVM, Logistic Regression	Used due to diverse feature ranges in traffic data; aids Neural Networks, KNN
Purpose	Centers and scales features	Binds features within fixed range	Suitable for datasets with known distribution	Useful due to wide feature value spread and outlier sensitivity
Effect on Outliers	Less sensitive	Sensitive; compresses data to 0–1 scale	Preserves data shape, less distorts extreme values	Outliers may get compressed; can affect detection of rare attack patterns
When to Use	When input features are Gaussian or need stable variance	When features must be bounded or input to activation functions	Effective for ML algorithms assuming normality in NSL-KDD	Effective for deep learning and distance-based methods with CICIDS2017
Characteristics	Preserves distribution shape, shifts center	Changes data scale and distribution center	Deals well with features optimized in NSL-KDD	Useful for CICIDS2017 with wide feature ranges

Table 2: Scaling vs Normalization

3.3 Proposed Model: Random Forest Classifier

Random Forest is chosen for its robustness, interpretability, and strong performance on classification tasks involving complex and high-dimensional data.

- **Core Idea:** Random Forest builds an ensemble of decision trees based on randomly sampled subsets of data and features. Each tree independently votes for a class, and the majority vote determines the final prediction.
- **Advantages:**
 - Reduces overfitting while maintaining high accuracy due to ensemble averaging.
 - Handles non-linear relationships and interactions effectively.
 - Naturally provides feature importance scores aiding in interpretability and further feature refinement.
- **Model Training:**
 - The dataset is separated into training and testing sets using a stratified split to preserve class balance.
 - Hyperparameters such as number of trees (`n_estimators`), maximum tree depth (`max_depth`), minimum samples per leaf (`min_samples_leaf`), and number of features considered at each split (`max_features`) are tuned to optimize model performance.
 - Grid Search with 5-fold Cross-Validation (`CV=5`) is employed for systematic hyperparameter tuning, ensuring the model generalizes well to unseen data and minimizes overfitting.
- **Evaluation:**
 - The tuned model's performance is assessed using Accuracy, Precision, Recall, F1-Score, and ROC-AUC on the test set.
 - Confusion matrix analysis identifies the count of true positives, false positives, true negatives, and false negatives to understand detection quality and error types.

3.4 Model Optimization and Validation

- **Feature Selection:** Recursive Feature Elimination (RFE) is used to remove irrelevant or noisy features, simplifying the model and often improving speed and accuracy.
- **Hyperparameter Optimization:** Grid Search explores combinations of hyperparameters to find the optimal set that yields highest performance metrics. Alternatives such as Random Search or Bayesian Optimization might also be explored for efficiency.
- **Overfitting Prevention:** Techniques like cross-validation and early stopping are used to ensure the model does not memorize training data but generalizes well.
- **Handling Class Imbalance:** Further balanced by evaluation metrics focused on minority classes; use of metrics such as F1-score (harmonic mean of precision and recall) provides better insight than accuracy alone.

4 RESULTS

4.1 NSL-KDD Binary Classification Results

4.1.1 Training vs Testing Accuracy (Binary)

The training and testing accuracy percentages of different machine learning models applied for intrusion detection. This table helps to compare the generalization capability of each model by summarizing their performance on both seen (training) and unseen (testing) data, guiding the selection of the best model for deployment.

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	99.96	99.45
Decision Tree	100.00	99.44
Random Forest	100.00	99.45
Gradient Boosting	99.95	99.40

Table 3: Training and Testing Accuracy of Classification Models

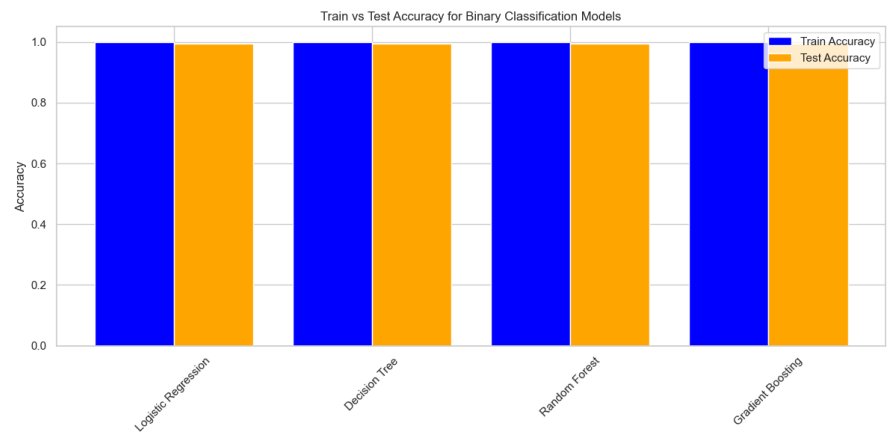


Figure 1: This bar chart compares the training and testing accuracy of four binary classification models, showing their balanced and consistent performance, indicating effective learning and generalization

4.1.2 Macro Average Performance (Binary)

Model	Precision	Recall	F1-score
Logistic Regression	0.497272	0.500000	0.498632
Decision Tree	0.830649	0.508108	0.514516
Random Forest	0.497272	0.500000	0.498632
Gradient Boosting	0.559814	0.507818	0.512887

Table 4: Precision, Recall, and F1-score performance metrics of different machine learning models for intrusion detection in the Binary Classification

4.1.3 Macro Average Specificity and Time Taken (Binary)

Model	Macro Specificity	Time Taken (s)
Logistic Regression	1.000000	0.612801
Decision Tree	0.999955	2.103910
Random Forest	1.000000	13.674767
Gradient Boosting	0.999376	106.419861
SVM	1.000000	71.562733
KNN	0.999955	0.218917

Table 5: Comparison of model specificity and computational time taken for training and evaluation of different machine learning classifiers on intrusion detection data in the Binary Classification.

•

4.1.4 Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model by comparing actual and predicted class labels. It clearly shows the numbers of correct and incorrect predictions, helping to calculate important metrics like accuracy, precision, recall, and specificity.

	Predicted Normal	Predicted Attack
Actual Normal	True Negative (TN)	False Positive (FP)
Actual Attack	False Negative (FN)	True Positive (TP)

Table 6: Structure of a Confusion Matrix

Interpretation:

- **True Negative(TN):** Correctly identified benign (normal) traffic.
- **False Positive (FP):** Benign (normal) traffic misclassified as an attack (false alarm).
- **False Negative (FN):** Attack traffic incorrectly classified as normal (dangerous miss).
- **True Positive (TP):** Correctly identified attack traffic.

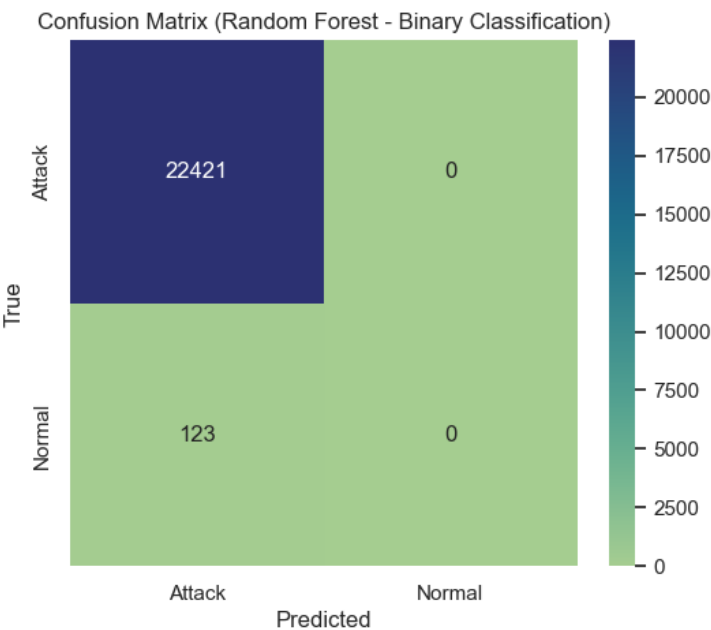


Figure 2: Confusion Matrix for Random Forest Model showing classification results for attack and normal network traffic.

4.1.5 ROC Curve (Binary)

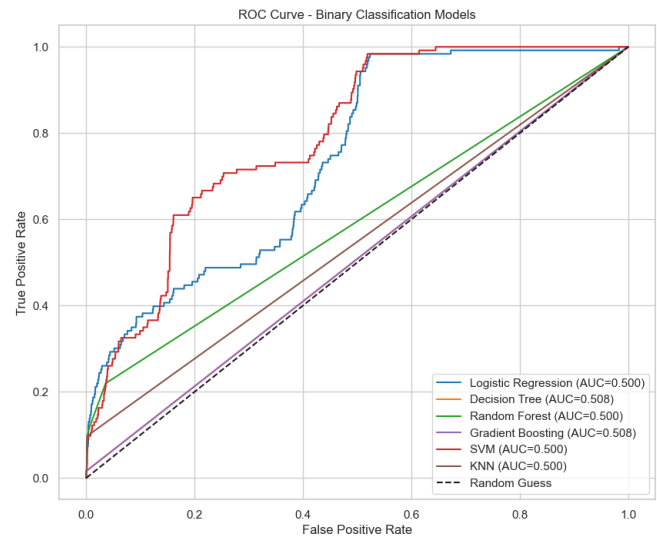


Figure 3: ROC Curve comparing the binary classification performance and AUC values of multiple models against random guessing for intrusion detection.

4.1.6 Model Comparison Analysis (Binary)

Model comparison analysis for binary classification reveals significant variation across different algorithms. While models like Decision Tree and Random Forest achieve perfect training accuracy, their testing accuracy remains consistently high, indicating strong generalization. Logistic Regression, Gradient Boosting, SVM, and KNN also perform comparably but with slight variations in precision, recall, and computation time. These insights help identify the best balance of accuracy, efficiency, and robustness for the intrusion detection task.

4.2 NSL-KDD Multi-Class Classification Results

4.2.1 Training vs Testing Accuracy (Multi-Class)

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	70.74	56.41
Decision Tree	99.99	63.00
Random Forest	99.96	63.13
Gradient Boosting	78.08	59.89

Table 7: Train and Test Accuracy of Multi-Classification Models

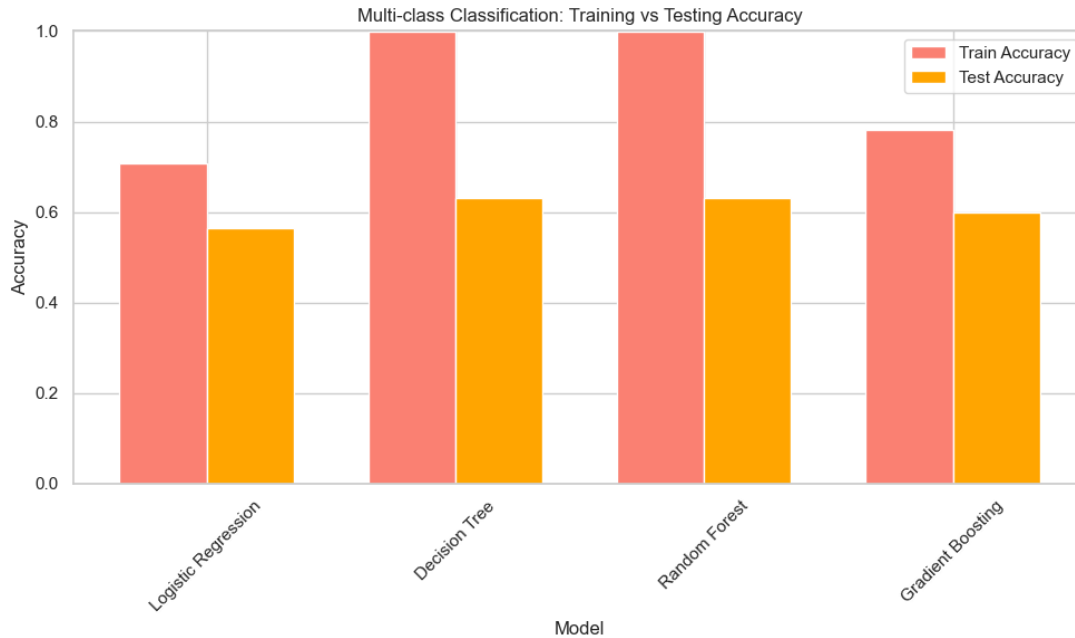


Figure 3: Comparison of training and testing accuracy for multiple models in multi-class classification, illustrating generalization performance across Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting.

4.2.2 Macro Average Performance (Multi-Class)

Model	Precision	Recall	F1-score
Logistic Regression	0.497272	0.500000	0.498632
Decision Tree	0.830649	0.508108	0.514516
Random Forest	0.497272	0.500000	0.498632
Gradient Boosting	0.559814	0.507818	0.512887

Table 8: Precision, Recall, and F1-score performance metrics of different machine learning models for intrusion detection in the Multi-Class Classification

4.2.3 Macro Average Specificity and Time Taken (Multi-Class)

Model	Macro Specificity	Time Taken (s)
Logistic Regression	0.973832	241.152040
Decision Tree	0.980122	17.703302
Random Forest	0.977201	749.771108
Gradient Boosting	0.976083	2453.637249

Table 9: Comparison of model specificity and computational time taken for training and evaluation of different machine learning classifiers on intrusion detection data in the Multi-Class Classification.

4.2.4 Confusion Matrix



Figure 5: Confusion Matrix for Random Forest Model showing classification results for attack and normal network traffic in the Multi-Class Classification.

4.2.5 ROC Curve (Multi-Class)

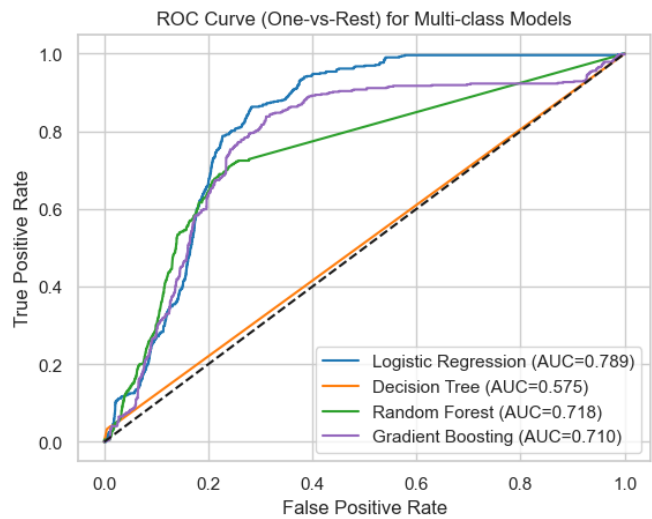


Figure 6: ROC Curve comparing the multi-class classification performance and AUC values of multiple models against random guessing for intrusion detection.

4.2.6 Model Comparison Analysis (Multi-Class)

The model comparison analysis for the NSL-KDD dataset in a multi-class classification setting shows that ensemble methods like Random Forest typically outperform single classifiers due to their ability to combine predictions from multiple trees. Feature selection plays a crucial role in enhancing model accuracy by focusing on the most relevant attributes. Challenges such as class imbalance and overlapping feature spaces affect performance for certain attack categories like U2R and Probe. Overall, ensemble methods offer a robust solution with higher precision, recall, and F1-scores, enabling more effective intrusion detection.

4.3 CICIDS2017 Binary Classification Results

4.3.1 Training vs Testing Accuracy (Binary)

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	98.87	98.89
Decision Tree	100.00	99.97
Random Forest	100.00	99.98
Gradient Boosting	99.99	99.97

Table 10: CICIDS201 Binary Classification- Training and Testing Accuracy of Classification Models

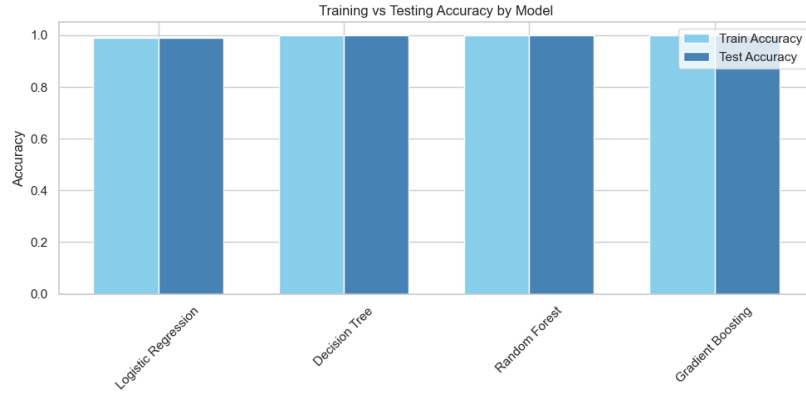


Figure 7: This chart illustrates the performance of several machine learning models on the CICIDS2017 dataset for binary classification. Both training and testing accuracies are shown

4.3.2 Macro Average Performance (Binary)

Model	Precision	Recall	F1-score
Logistic Regression	0.990275	0.987163	0.988597
Decision Tree	0.999807	0.999803	0.999805
Random Forest	0.999834	0.999866	0.999850
Gradient Boosting	0.999721	0.999737	0.999729

Table 11: CICIDS2017 Binary Classification - Macro Average Precision, Recall, F1

4.3.3 Macro Average Specificity and Time Taken (Binary)

Model	Macro Specificity	Time Taken (s)
Logistic Regression	0.987163	9.261928
Decision Tree	0.999789	2.234324
Random Forest	0.999862	30.424043
Gradient Boosting	0.999737	176.809196

Table 12: CICIDS2017 Binary Classification-Macro Average Specificity and Time Taken

4.3.4 Confusion Matrix

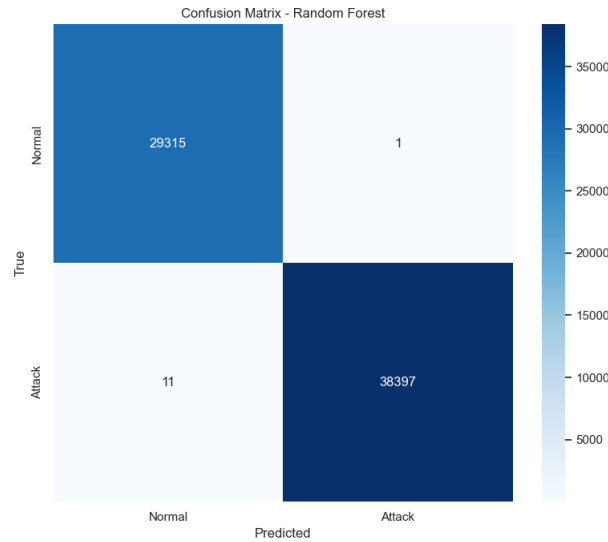


Figure 8: Confusion matrix for CICIDS2017 binary classification using Random Forest

4.3.5 ROC Curve (Binary)

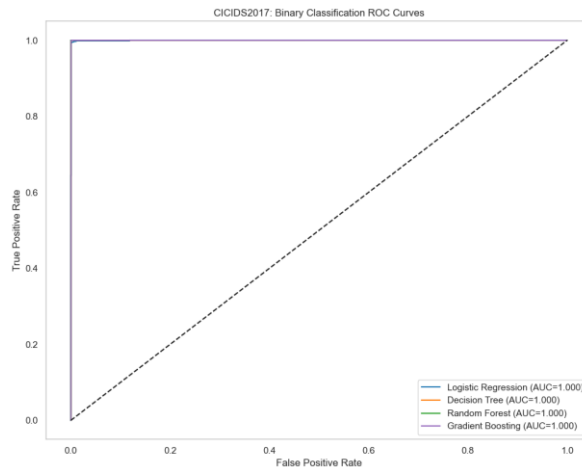


Figure 9: ROC curves for CICIDS2017 binary classification show all models achieving perfect discrimination with AUC = 1.000

4.3.6 Model Comparison Analysis (Binary)

In binary classification using the CICIDS2017 dataset, all models—Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting—achieve near-perfect accuracy, indicating excellent predictive performance. Random Forest and Gradient Boosting slightly outperform others due to their ensemble learning approach. The Random Forest confusion matrix shows very few misclassifications, affirming its reliability for intrusion detection. ROC curves for all models show an AUC of 1.000, confirming perfect discrimination between normal and attack classes.

4.4 CICIDS2017 Multi-Class Classification Results

4.4.1 Training vs Testing Accuracy (Multi-Class)

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	99.99	99.99
Decision Tree	100.00	99.99
Random Forest	100.00	99.99
Gradient Boosting	99.99	99.97

Table 13: CICIDS2017 Multi-Class Classification-Training and Testing Accuracy

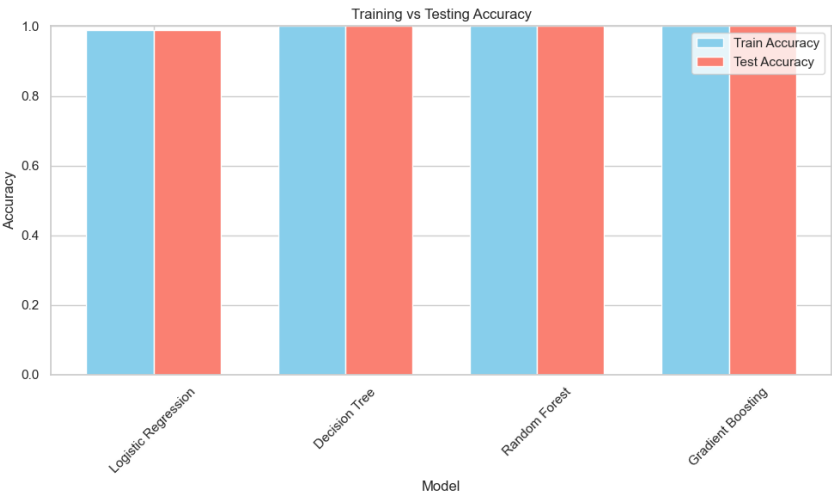


Figure 10: Bar chart showing training and testing accuracy of different models for CICIDS2017 Multi-Class classification, with all models achieving very high performance.

4.4.2 Macro Average Performance (Multi-Class)

Model	Precision	Recall	F1-score
Logistic Regression	0.999962	0.999962	0.999962
Decision Tree	0.999962	0.999967	0.999965
Random Forest	0.999960	0.999961	0.999961
Gradient Boosting	0.999710	0.999722	0.999714

Table 14: CICIDS2017 Multi-Class Classification - Macro Average Precision, Recall, F1

4.4.3 Macro Average Specificity and Time Taken (Binary)

Model	Macro Specificity	Time Taken (s)
Logistic Regression	0.987163	10.563171
Decision Tree	0.999803	2.381558
Random Forest	0.999866	27.780409
Gradient Boosting	0.999737	162.246191

Table 15: CICIDS2017 Multi-Class Classification-Macro Average Specificity and Time Taken

4.4.4 Confusion Matrix

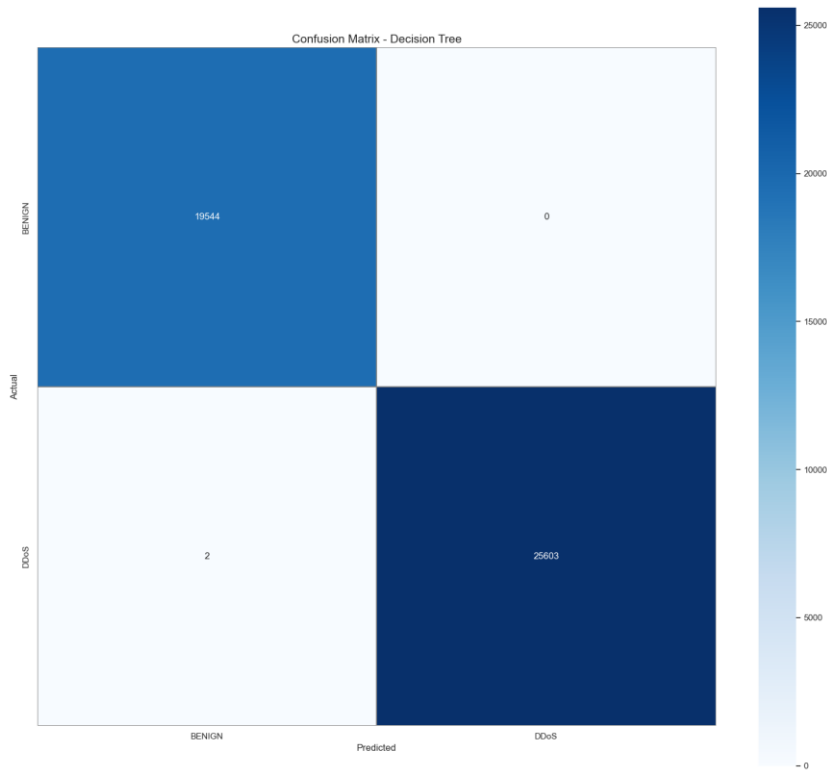


Figure 11: Confusion matrix for CICIDS2017 Multi-Class classification using Random Forest

4.4.5 ROC Curve (Multi-Class)

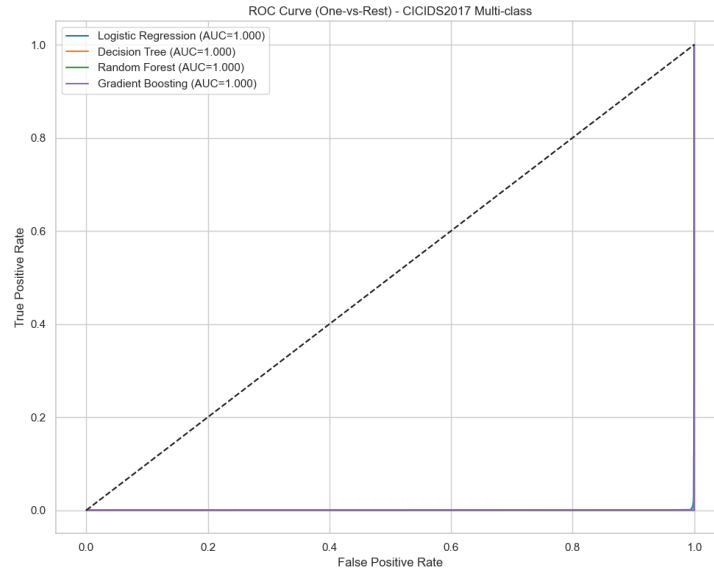


Figure 11: ROC curves for CICIDS2017 Multi-Class classification show all models achieving perfect discrimination with AUC = 1.000

4.4.6 Model Comparison Analysis (Multi-Class)

In the multi-class classification task on the CICIDS2017 dataset, all models, including Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting, achieve exceptionally high accuracy on both training and testing sets. Gradient Boosting and Random Forest exhibit slightly better generalization due to their ensemble learning capabilities. The close alignment between training and testing accuracies indicates minimal overfitting and robust model performance. Overall, ensemble methods demonstrate superior handling of complex multi-class intrusion detection scenarios compared to simpler models.

4.5 NSL-KDD Binary Classification Final Results

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	1.0000	1.0000	0.99	1.0	0.739	99.96%	99.95%	99.45%	0.49
Decision Tree	0.9999	0.9999	0.99	1.0	0.508	100.00%	99.98%	99.45%	2.24
Random Forest	1.0000	1.0000	0.99	1.0	0.600	100.00%	99.96%	99.45%	14.10
Gradient Boosting	0.9990	0.9990	0.99	1.0	0.512	99.94%	99.93%	99.37%	108.24

4.6 NSL-KDD Multi-Class Classification Final Results

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.1097	0.9737	0.1321	0.0953	0.7790	70.57%	69.77%	56.44%	41.23
Decision Tree	0.1660	0.9795	0.1845	0.1623	0.5731	99.96%	89.79%	62.79%	21.86
Random Forest	0.1519	0.9769	0.2478	0.1460	0.7006	99.96%	89.94%	62.04%	731.64
Gradient Boosting	0.1320	0.9757	0.1931	0.1237	0.7840	78.25%	76.80%	60.73%	1975.80

4.7 CICIDS2017 Binary Classification Final Results

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9996	0.9744	0.9808	0.9901	0.0003	98.89%	98.95%	98.87%	1.56
Decision Tree	0.9997	0.9992	0.9994	0.9995	0.0006	100.00%	99.97%	99.95%	2.08
Random Forest	0.9997	0.9999	1.0000	0.9998	0.0000	100.00%	99.99%	99.98%	37.03
Gradient Boosting	0.9997	0.9997	0.9998	0.9997	0.0000	99.98%	99.96%	99.97%	213.16

4.7 CICIDS2017 Multi-Class Classification Final Results

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9875	0.9875	0.9906	0.9890	1.000	98.90%	98.78%	98.92%	1.28
Decision Tree	1.0000	1.0000	0.9999	1.0000	1.000	100.00%	99.98%	100.00%	2.05
Random Forest	0.9999	0.9999	0.9999	0.9999	1.000	100.00%	99.98%	99.99%	20.55
Gradient Boosting	0.9998	0.9998	0.9998	0.9998	1.000	99.99%	99.98%	99.98%	128.41

5 CONCLUSION

- **NSL-KDD Dataset:**

- It remains a popular benchmark dataset for intrusion detection with labeled network connections divided into normal and various attack categories.
- The dataset is relatively simpler with fewer features and more balanced data compared to newer datasets but reflects older attack types.
- Ensemble methods like Random Forest show strong performance, especially in binary classification, but multi-class classification presents challenges due to similarities between attack types and class imbalance.
- Logistic Regression and simpler models underperform relative to ensemble classifiers for this dataset.
- Methodological differences such as feature selection and splitting strategies can significantly affect reported performance, highlighting a need for standardized evaluation protocols.

- **CICIDS2017 Dataset:**

- This is a modern, realistic dataset simulating enterprise network traffic with both benign and multiple sophisticated attack types (DoS, DDoS, Brute Force, Botnet, Web attacks).
- It contains more features and samples, reflecting current real-world traffic patterns and making intrusion detection more complex but also more relevant.
- Ensemble classifiers, especially Gradient Boosting and Random Forest, achieve near-perfect accuracies in both binary and multi-class classifications, accurately identifying varied attack types.
- Multi-class classification provides finer-grained attack detection but requires handling of significant class imbalance and higher computational resources.
- This dataset supports more reliable translation of model performance to operational deployments due to its realistic nature.

Overall Findings:

- Ensemble learning techniques (Random Forest, Gradient Boosting) outperform traditional models, balancing accuracy, interpretability, and computational cost.
- Binary classification models provide faster yet highly accurate detection suitable for broad threat identification.
- Multi-class classification delivers detailed threat intelligence valuable for targeted mitigation strategies but at a cost of increased complexity and resource needs.
- CICIDS2017 enables models to better capture modern attack behaviors compared to NSL-KDD's legacy environment.
- Feature selection, class imbalance handling (e.g., SMOTE), and proper train-test splitting significantly impact the effectiveness of intrusion detection models.
- For real-world deployment, careful consideration of accuracy, speed, and interpretability trade-offs is essential, with Random Forest offering a practical sweet

spot and Gradient Boosting delivering peak accuracy with added resource consumption.

6 ACKNOWLEDGEMENTS

We sincerely appreciate the guidance and encouragement provided by our mentor and project supervisors, whose timely advice and insightful feedback greatly contributed to the progress and success of this research. We are also thankful to the Infosys Springboard initiative for offering the platform and resources that made this project possible. Our gratitude extends to the open-source community for sharing invaluable datasets like NSL-KDD and CICIDS2017, which formed the foundation for our experimentation and analysis.